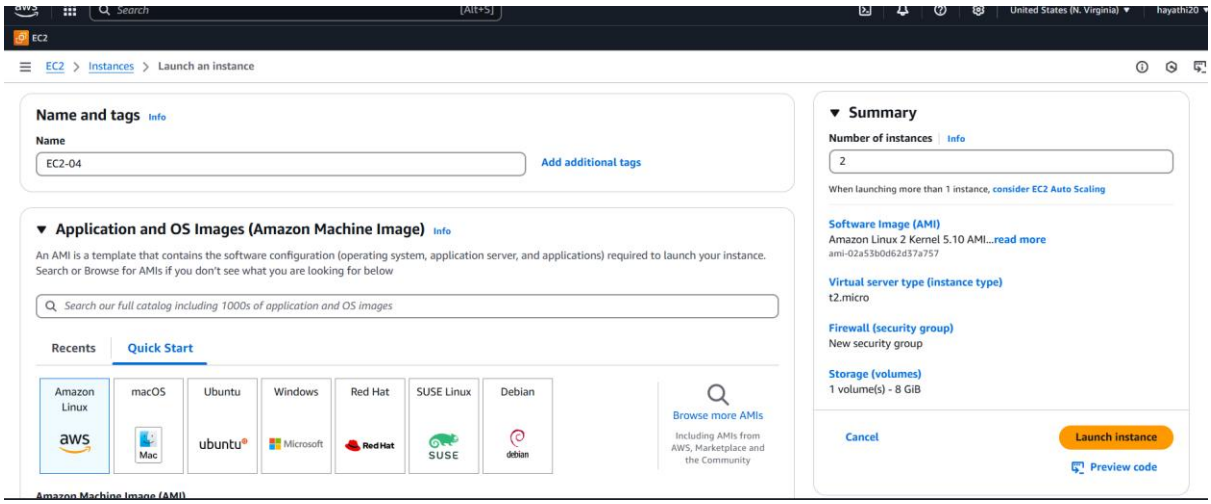
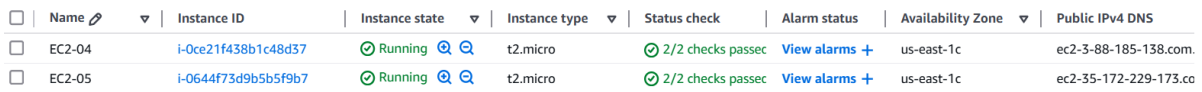
Task 6- LAMBDA & EFS

Task 6.a) How to start and stop multiple ec2 instances in single function?

Step1: Launched 2 EC2 instances



Here Instances are successfully launched in same region

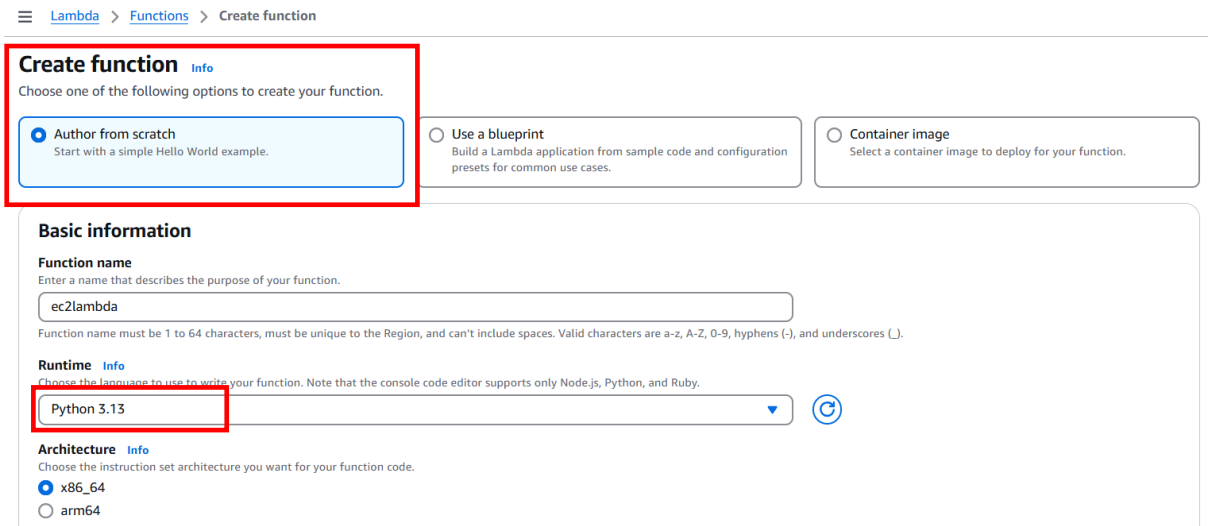


Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
EC2-04	i-0ce21f438b1c48d37	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1c	ec2-3-88-185-138.com.
EC2-05	i-0644f73d9b5b5f9b7	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1c	ec2-35-172-229-173.co

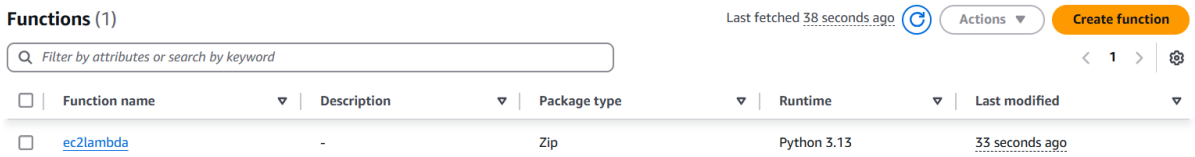
Step 2: Go to Lambda → Create function

Select Author from scratch

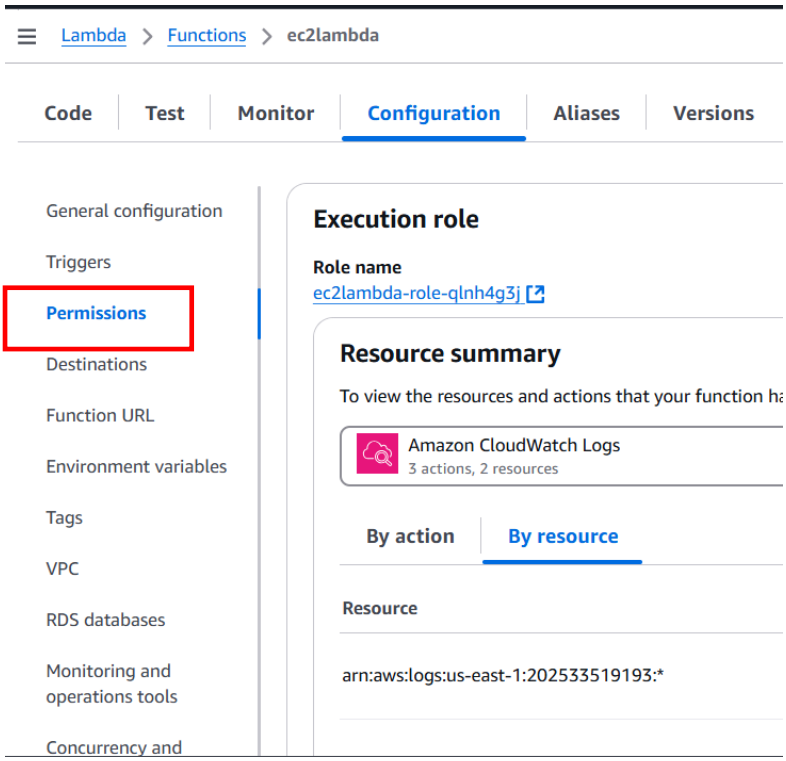
Runtime code-> Python



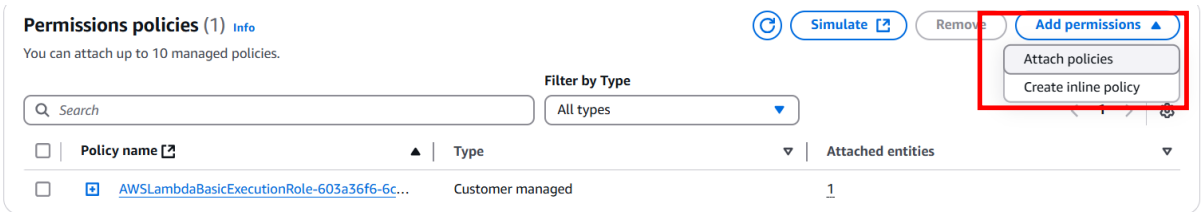
Here the Function is created successfully



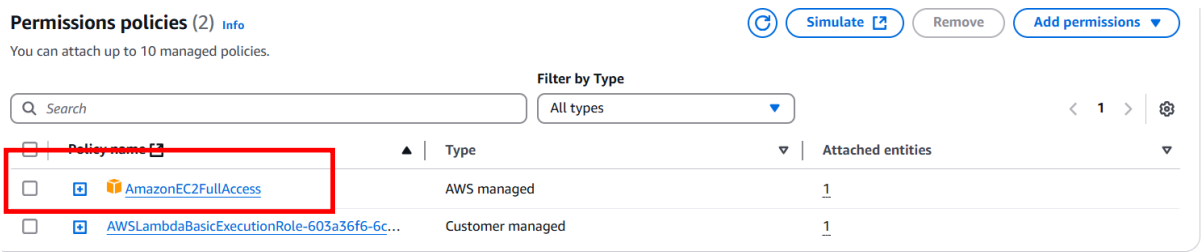
Open newly created function and go to permission-> click on Execution role



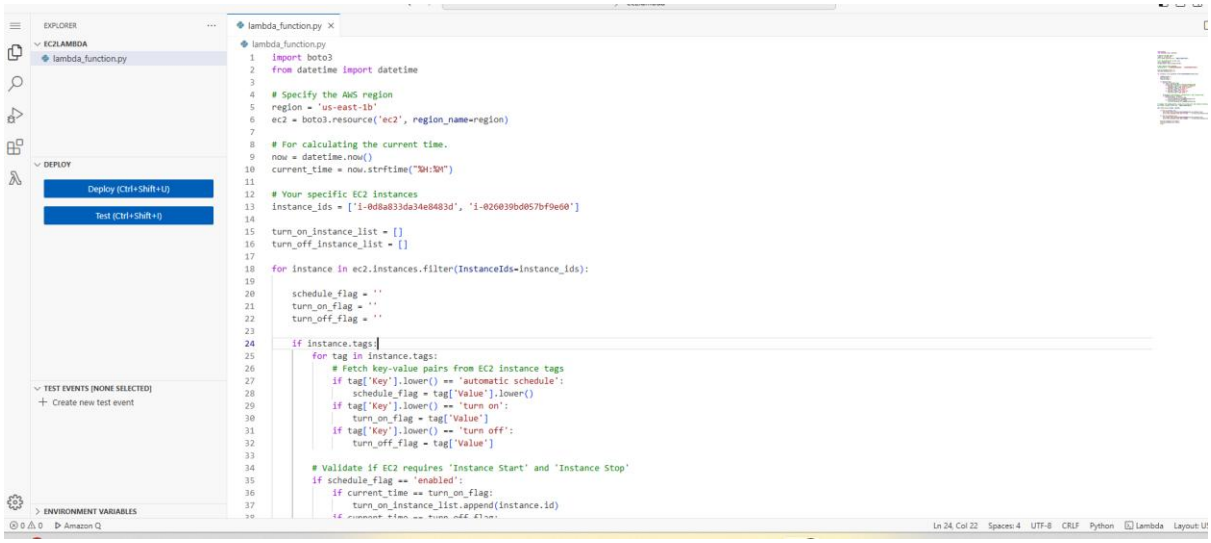
Now add permission -> Attach EC2 policy full access



Now EC2 full access is provided for this role



Goto the Lambda newly created function and click on code to write our own code in python



CODE:

```
import boto3
from datetime import datetime

# Specify the AWS region
region = 'us-east-1b'
ec2 = boto3.resource('ec2', region_name=region)

# For calculating the current time.
now = datetime.now()
current_time = now.strftime("%H:%M")

# Your specific EC2 instances
instance_ids = ['i-0d8a833da34e8483d', 'i-026039bd057bf9e60']

turn_on_instance_list = []
turn_off_instance_list = []

for instance in ec2.instances.filter(InstanceIds=instance_ids):

    schedule_flag = ''
    turn_on_flag = ''
    turn_off_flag = ''

    if instance.tags:
        for tag in instance.tags:
            # Fetch key-value pairs from EC2 instance tags
            if tag['Key'].lower() == 'automatic schedule':
                schedule_flag = tag['Value'].lower()
            if tag['Key'].lower() == 'turn on':
                turn_on_flag = tag['Value']
            if tag['Key'].lower() == 'turn off':
```

```
            turn_off_flag = tag['Value']

        # Validate if EC2 requires 'Instance Start' and 'Instance Stop'
        if schedule_flag == 'enabled':
            if current_time == turn_on_flag:
                turn_on_instance_list.append(instance.id)
            if current_time == turn_off_flag:
                turn_off_instance_list.append(instance.id)

# Triggers the Lambda_handler, passes EC2 instance ID, and executes Instance_state operation.
ec2_client = boto3.client('ec2', region_name=region)

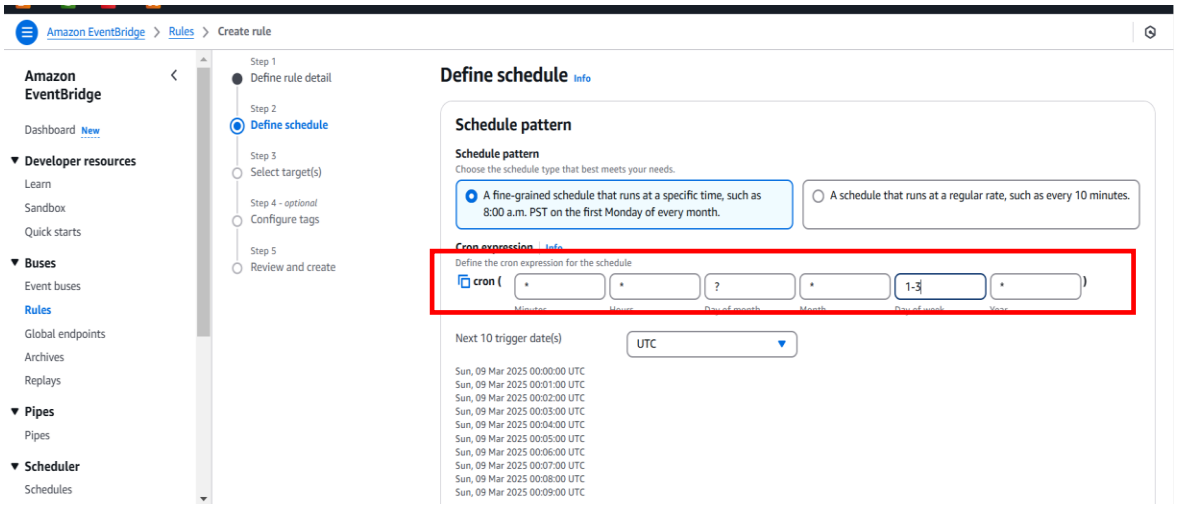
def lambda_handler(event, context):

    if turn_on_instance_list:
        ec2_client.start_instances(InstanceIds=turn_on_instance_list)
        print('EC2 Instances have been started: ' + str(turn_on_instance_list))

    if turn_off_instance_list:
        ec2_client.stop_instances(InstanceIds=turn_off_instance_list)
        print('EC2 Instances have been stopped: ' + str(turn_off_instance_list))

    turn_on_instance_list.clear()
    turn_off_instance_list.clear()
    exit()
```

Now adding the trigger to schedule the event

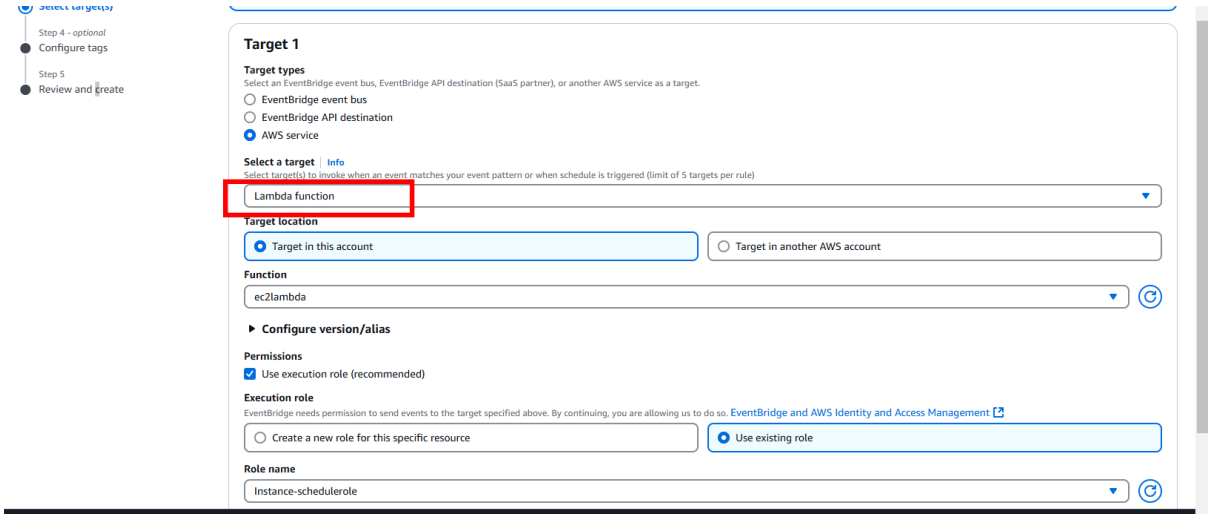


This cron expression will trigger **every minute**, but **only on Monday, Tuesday, and Wednesday**, regardless of the month or year. The function will check the EC2 instances and take action based on their tags (Turn On and Turn Off times).

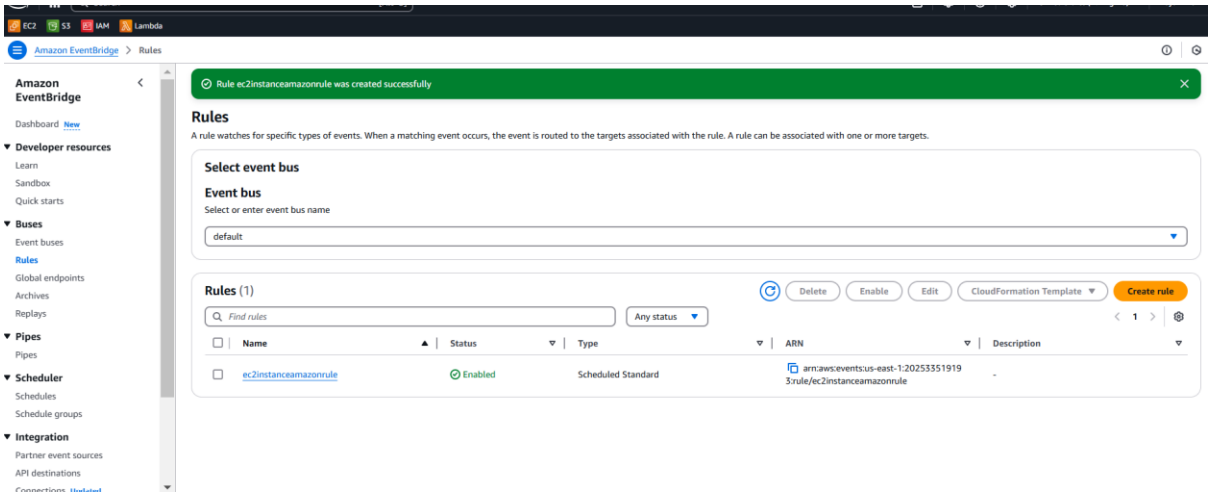
How does it work?

- If today is **Monday** and the time matches the Turn On tag for an instance, it will be started.
- If today is **Wednesday** and the time matches the Turn Off tag, the instance will be stopped.
- On **Thursday to Sunday**, the cron job does **not** run, so the instances remain in their last state.

Click on next and select Target

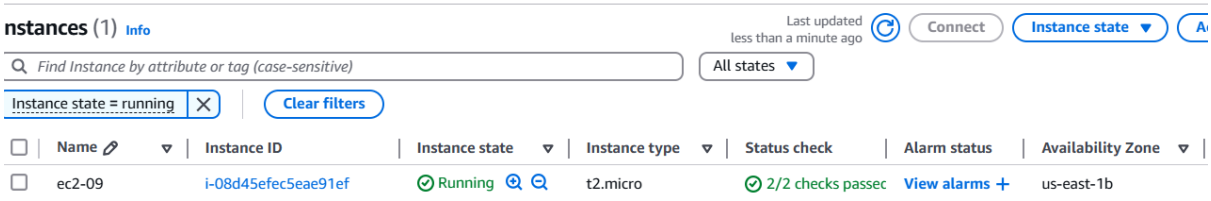


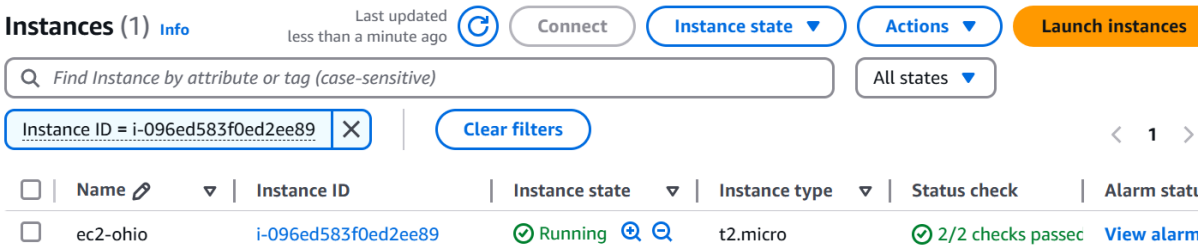
Now Amazon Event bridge rule is created successfully



Task 6. B) Delete the instances on multiple locations

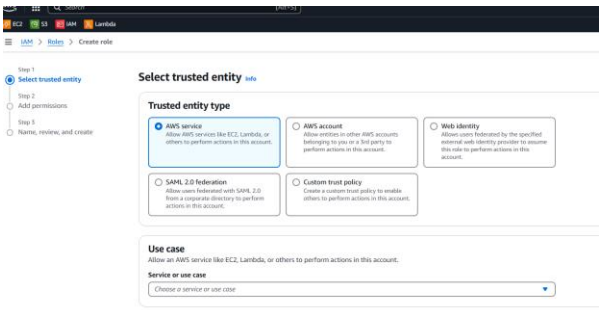
Created the EC2 instances in two regions (N.virgina and Ohio)



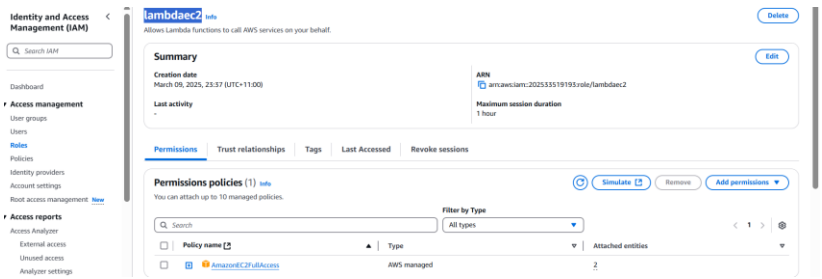


Create an IAM Role for Lambda

1. Go to the **AWS IAM Console** → **Roles** → **Create Role**.
2. Select **AWS Service** → **Lambda**.



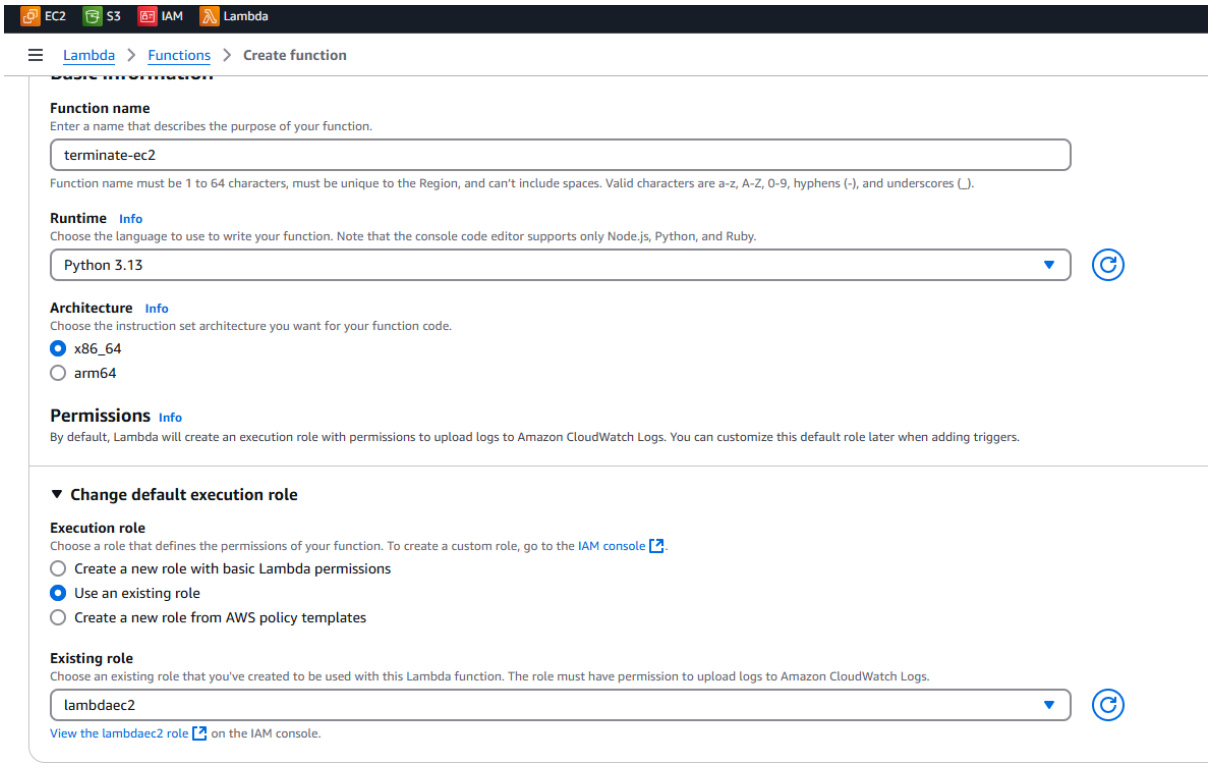
3. Click **Next** and attach the following policies: **AmazonEC2FullAccess**
4. Click **Next**, name the role **lambdaec2**, and create the role.



Creation of AWS Lambda Function

Open AWS Lambda Console

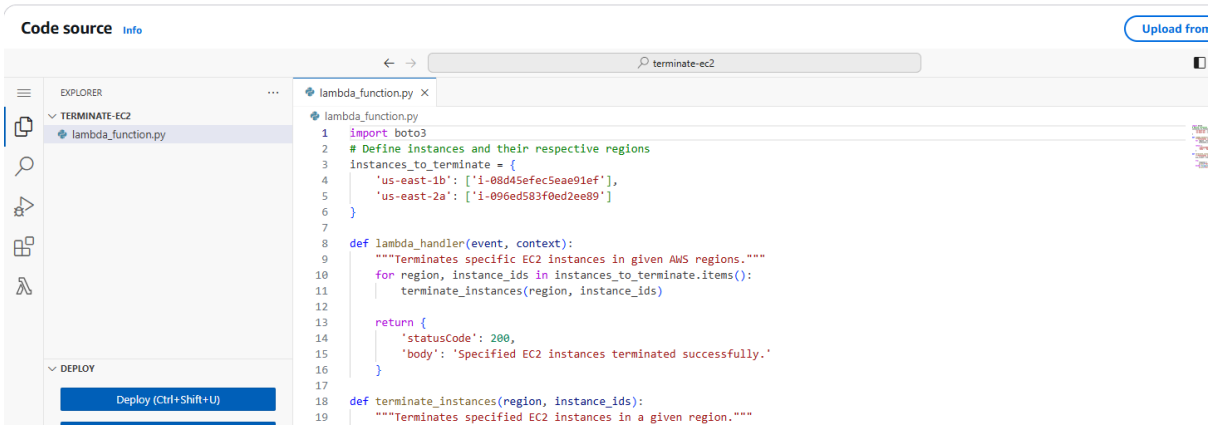
1. Go to AWS Lambda Console
2. Click **Create Function**.
3. Choose **Author from scratch**.
4. Set the **Function name** to terminate-ec2.
5. **Runtime**: Choose **Python 3.9 or later**.
6. **Execution Role**: Select **"Use an existing role"** and choose **lambdaec2**.
7. Click **Create Function**.



Add Python Code in AWS Lambda

Go to code in the created lambda function and write the python code

Click on Deploy



Verified the output by clicking on Test and received the below response

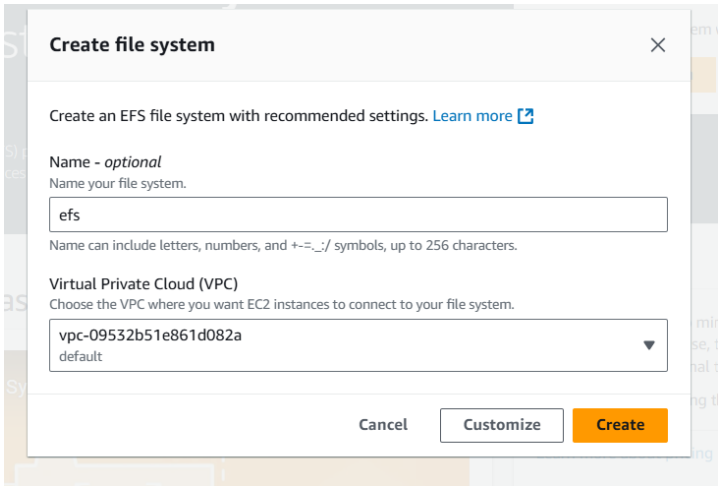


Instances are teminated

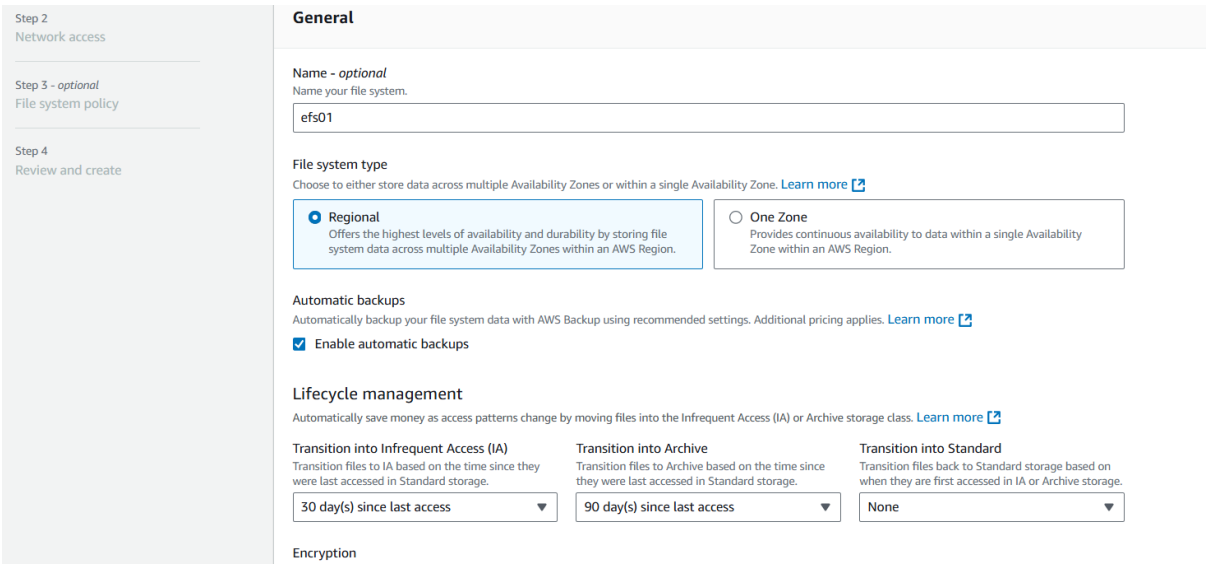


Task 6. C) EFS

Goto EFS-> Create file system

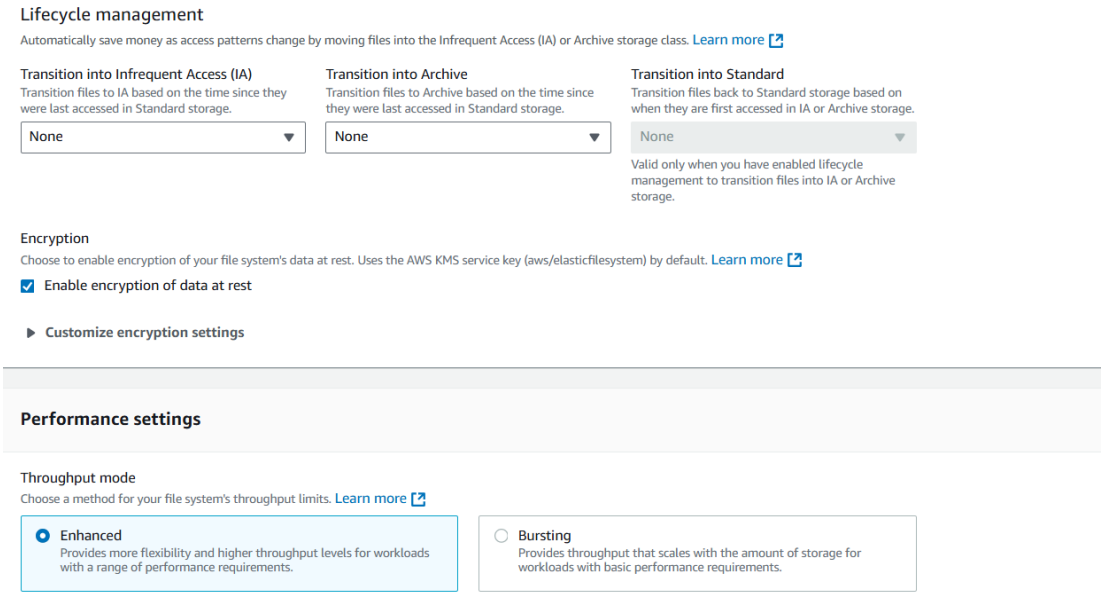


Click on customer to go to general settings



In the below, select None on Lifecycle management

Disable the tick in Encryption, we can enable it if needed based on requirements.

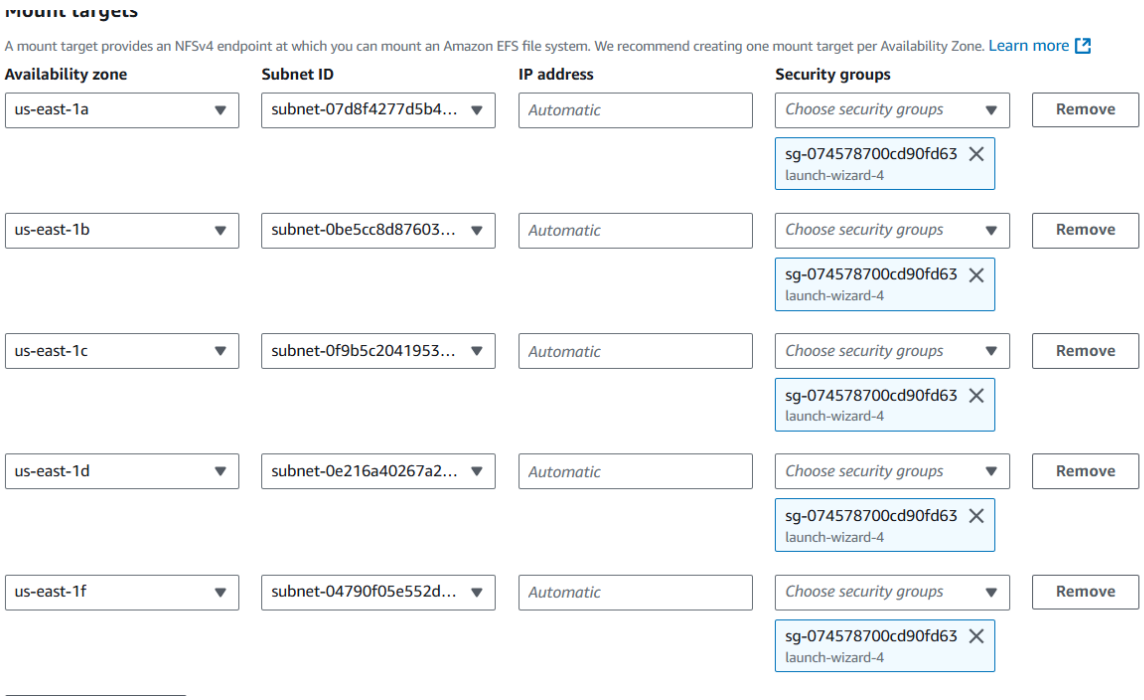


In the performance settings, select Enhanced->Bursting for standard throughput

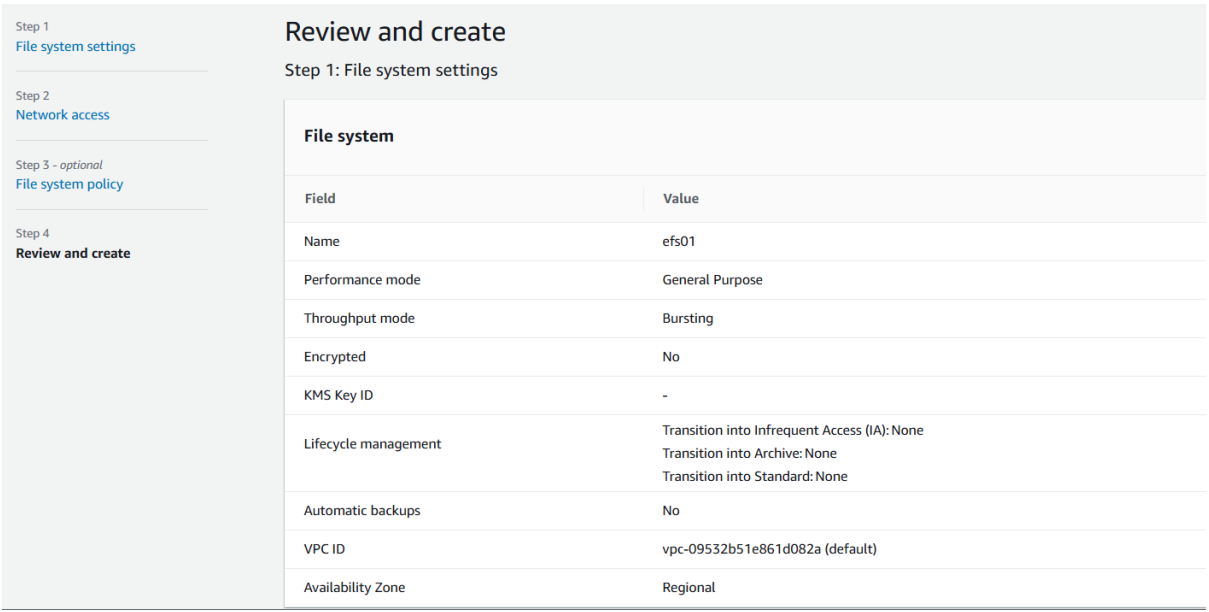
Next from the network setting choose the default VPC



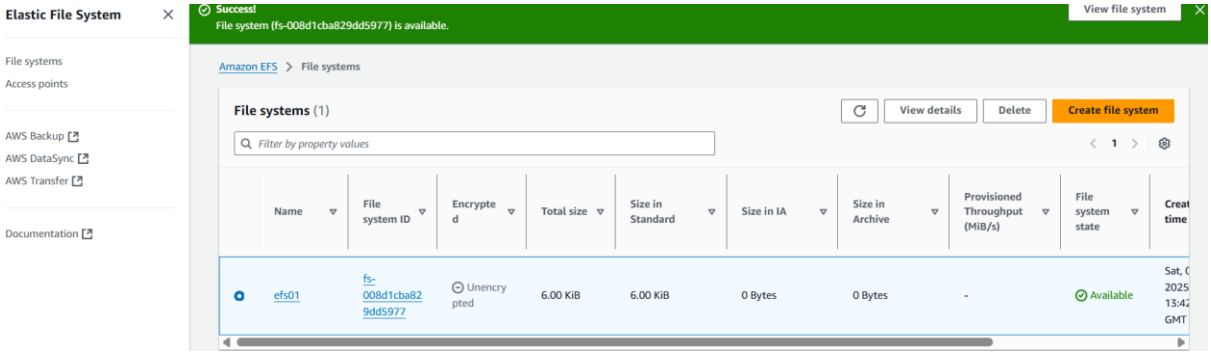
Remove all the default security groups and then copy and paste the security group from EC2 because in the default NFS is not enabled.



Click on next -> create

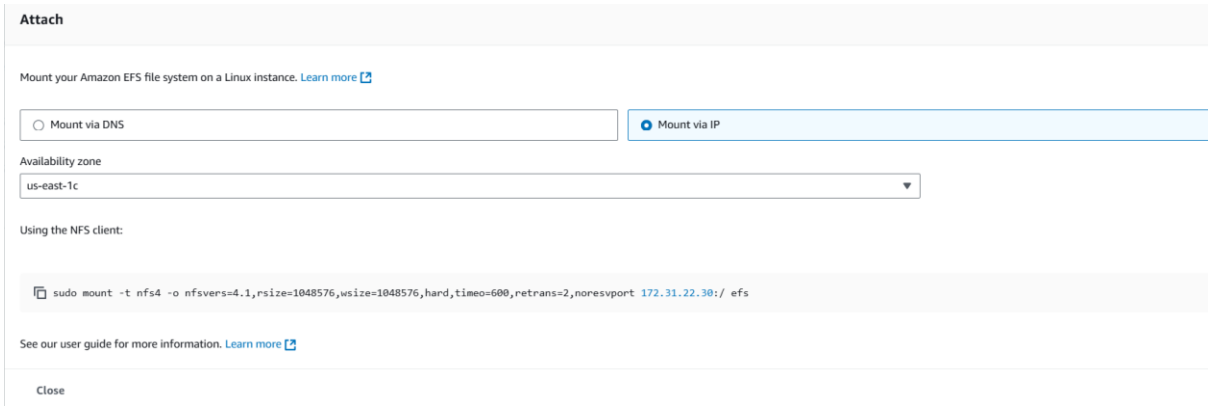


Successfully created filesystem



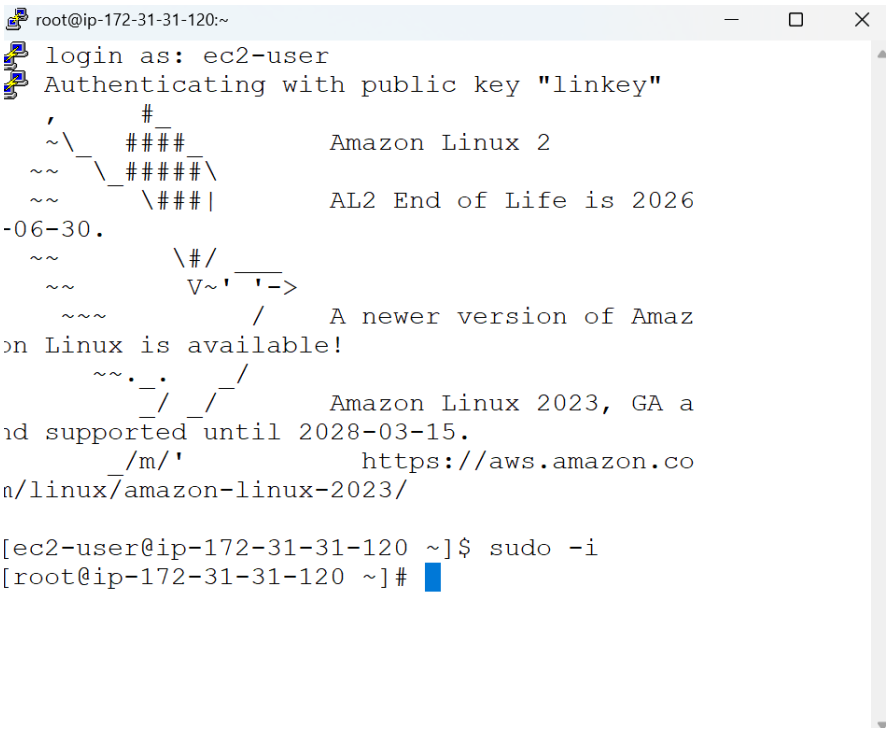
Now click on attach-> two options available -> select attach via IP

Choose server availability zone



Now connect with putty

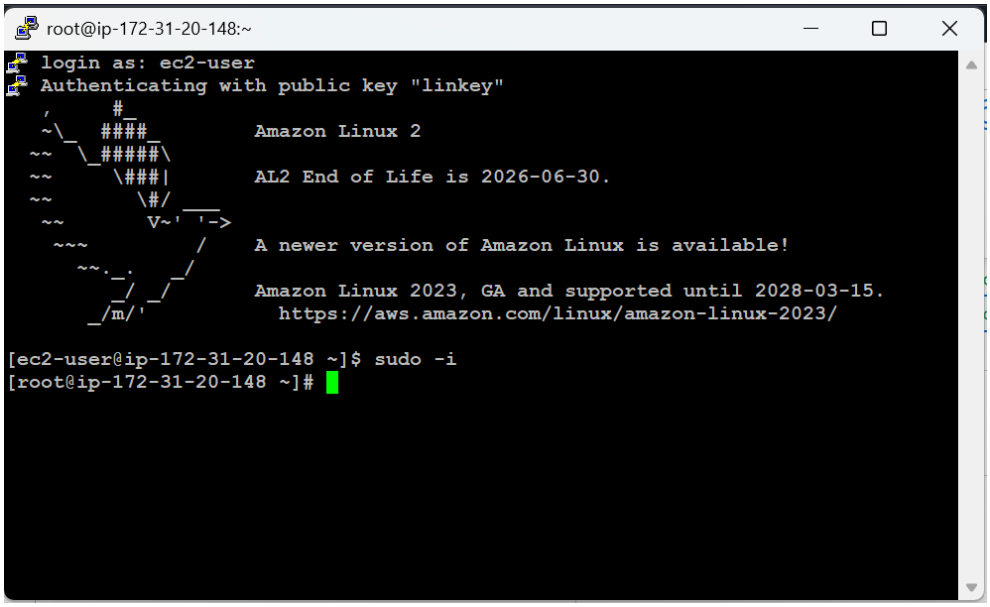
Here for Instance1(EFS-01) connected with putty as root user



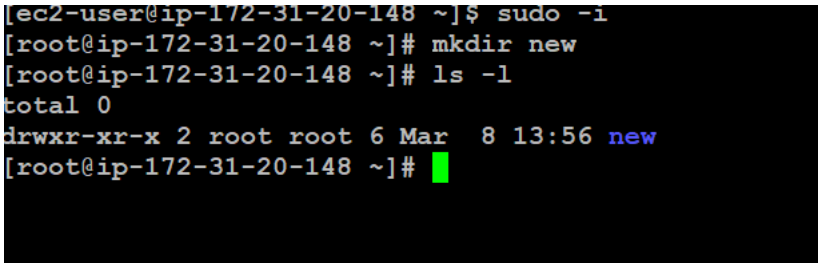
Next create a new directory to mount

```
[ec2-user@ip-172-31-31-120 ~]$ sudo -i
[root@ip-172-31-31-120 ~]# mkdir new
[root@ip-172-31-31-120 ~]# ls -l
total 0
drwxr-xr-x 2 root root 6 Mar  8 13:54 new
[root@ip-172-31-31-120 ~]#
```

Repeating the same step for connecting instance02(EFS-02) and connected in putty as root user



In the EFS-02 also create the new directory



Now two servers are ready with two directories to mount the NFS client.

Using the NFS client:

```
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport 172.31.22.30:/ efs
```

Mount the above command in servers

In the place of efs change it to created directory name and it is successful in both servers

```
rwxr-xr-x 2 root root 6 Mar  8 13:54 new
root@ip-172-31-31-120 ~]# sudo mount -t nfs4 -o nfsver
=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retran
=2,noresvport 172.31.22.30:/ new
root@ip-172-31-31-120 ~]#
```

```
total 0
drwxr-xr-x 2 root root 6 Mar  8 13:56 new
[root@ip-172-31-20-148 ~]# sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsiz
=1048576,hard,timeo=600,retrans=2,noresvport 172.31.22.30:/ new
[root@ip-172-31-20-148 ~]#
```

Use command cd new/ and ll to check if anything is available but it is empty inside in both servers

```
root@ip-172-31-31-120:~/new
   ~~        V~' '->
    ~~~           /    A newer version of Amaz
on Linux is available!
      ~~._.    _/
         _/ _/       Amazon Linux 2023, GA a
id supported until 2028-03-15.
       _/m/'           https://aws.amazon.co
m/linux/amazon-linux-2023/

ec2-user@ip-172-31-31-120 ~]$ sudo -i
root@ip-172-31-31-120 ~]# mkdir new
root@ip-172-31-31-120 ~]# ls -l
otal 0
drwxr-xr-x 2 root root 6 Mar  8 13:54 new
root@ip-172-31-31-120 ~]# sudo mount -t nfs4 -o nfsver
=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retran
=2,noresvport 172.31.22.30:/ new
root@ip-172-31-31-120 ~]# cd new
root@ip-172-31-31-120 new]# ls
root@ip-172-31-31-120 new]# cd new/
bash: cd: new/: No such file or directory
root@ip-172-31-31-120 new]# ll
otal 0
root@ip-172-31-31-120 new]#
```

Now I created one file named test in EFS-01 using command touch test (filename) and this newly created file can be listed in EFS-02 also

```
[root@ip-172-31-31-120 new]# ll
total 0
[root@ip-172-31-31-120 new]# touch test
[root@ip-172-31-31-120 new]#
```

```
-bash: 11: command not found
[root@ip-172-31-20-148 new]# ll
total 0
[root@ip-172-31-20-148 new]# ls
test
[root@ip-172-31-20-148 new]#
```

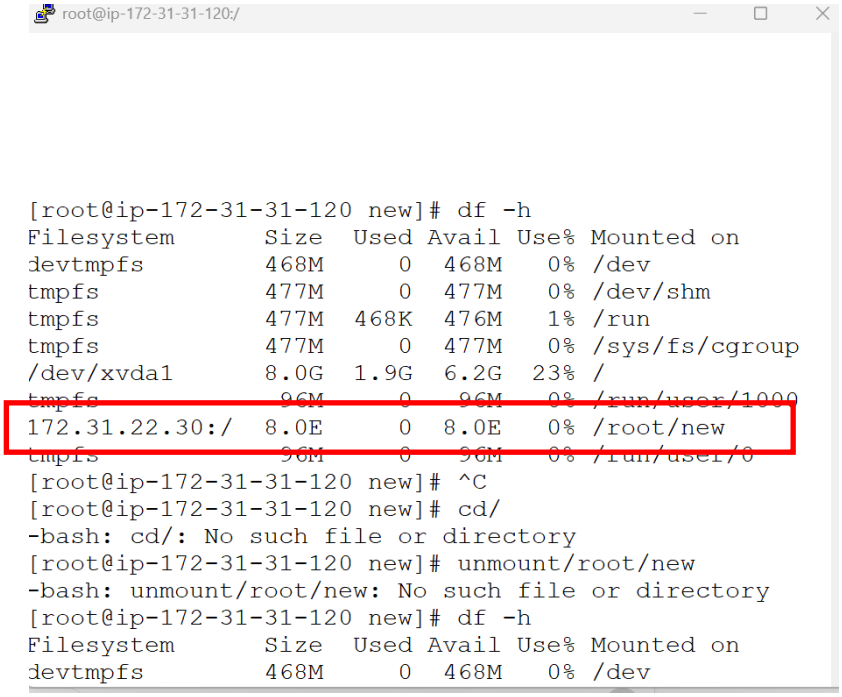
Here in this server, I have not given any command to create the file, but still I could see the file which is created in another instance. Because of the same elastic file system in the directory which shares through network.

```
root@ip-172-31-20-148 new]# ll
otal 8
rwxr-xr-x 2 root root 6144 Mar  8 14:14 efs_test
rw-r--r-- 1 root root    0 Mar  8 14:07 test
root@ip-172-31-20-148 new]# ls -l
otal 12
rwxr-xr-x 2 root root 6144 Mar  8 14:14 efs_test
rw-r--r-- 1 root root   37 Mar  8 14:17 newfile
rw-r--r-- 1 root root    0 Mar  8 14:07 test
root@ip-172-31-20-148 new]# cat newfile
i  EFS1...
```

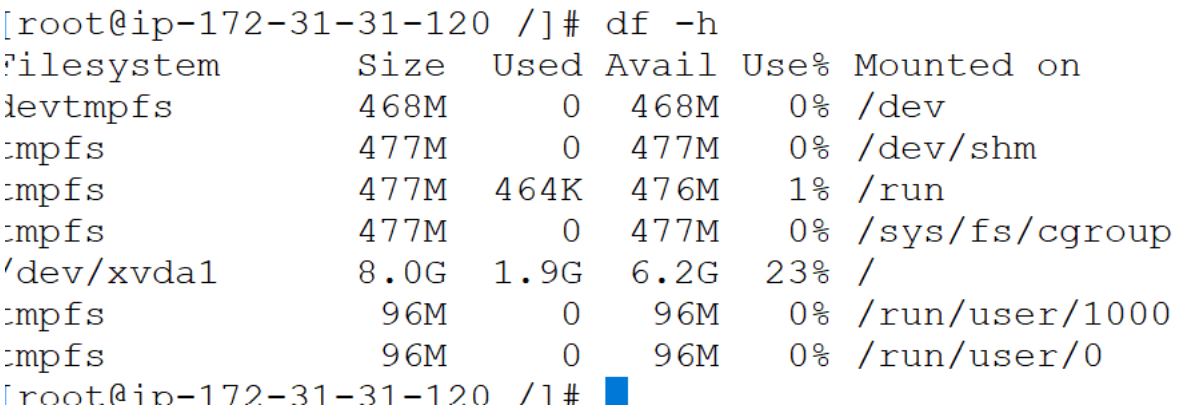
Here tried adding file and some texted in EC2 and same thing is reflected in EC1 also

Unmounting the NFS

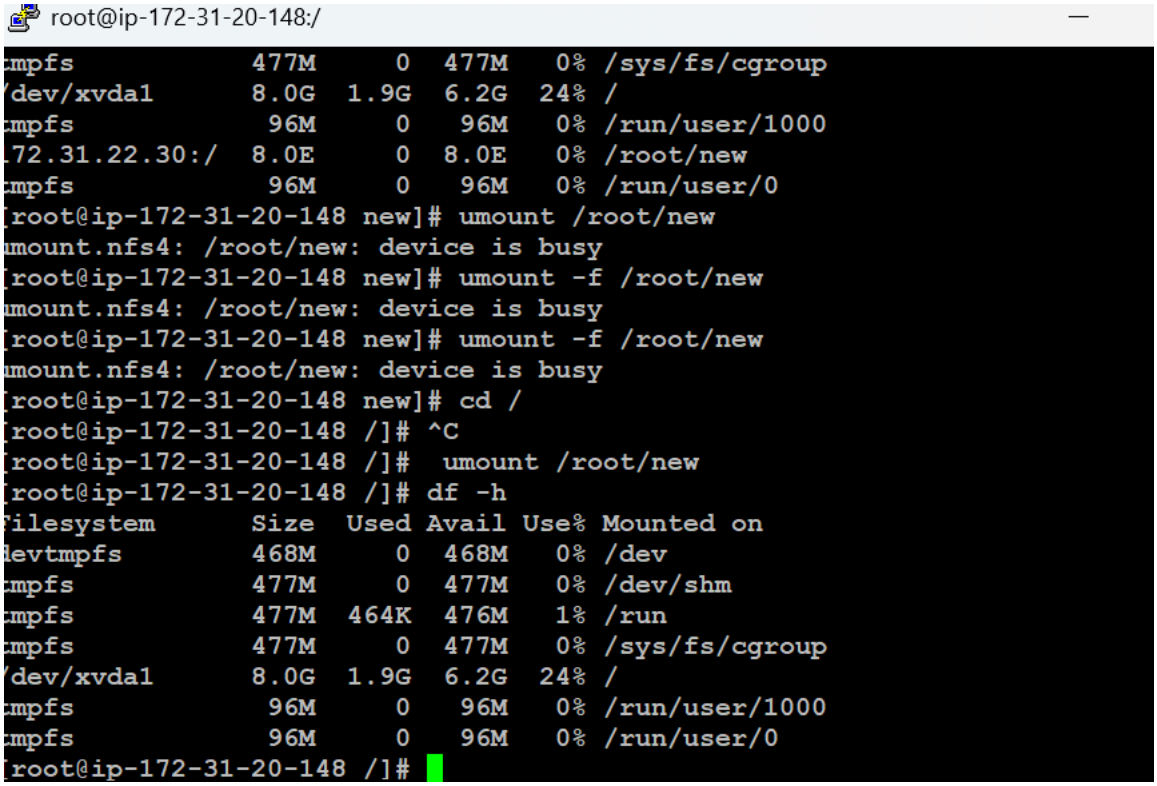
- Run the following command to check where the file system is mounted: **df -h**
- Ensure you're not inside the mounted directory: **cd /**
- To unmount the file system mounted at /root/new, run the following command: **umount /root/new**
- Here I used umount /root/new because my file system is mounted here and instance IP address is highlighted .



Confirming the unmount by giving df -h command and now my ip address is not present here and it is successful.



Same step is followed in other instance to unmount and it is successful



I also verified by using the ls command to check if my newly added files, named test and efs_test, were present. Since they were not found, the unmount was successful.

